



01 PAO25-25 - PYTHON, Data Types



Luis Pilaguano

Diagnóstico de cáncer de mama con el Perceptrón}

1 Caso Práctico: Neurona de McCulloch y Pitts

1.1 2. Aplicando el MPNeuron a un caso práctico real

1.1.1 2.1. Conjunto de datos

Esta es una copia de los conjuntos de datos de UCI ML Breast Cancer Wisconsin (Diagnóstico). <https://goo.gl/U2Uwz2> Las características de entrada se calculan a partir de una imagen digitalizada de un aspirado de aguja fina (FNA) de una masa mamaria. Describe las características de los núcleos celulares presentes en la imagen. El plano de separación descrito anteriormente se obtuvo utilizando el método de árbol de múltiples superficies (MSM-T) [KP Bennett, "Construcción de un árbol de decisión mediante programación lineal". Proceedings of the 4th Midwest Artificial Intelligence and Cognitive Science Society, pp. 97101, 1992], un método de clasificación que utiliza la programación lineal para construir un árbol de decisión. Los rasgos relevantes se seleccionan mediante una búsqueda exhaustiva en el espacio de 1-4 rasgos y 1-3 planos de separación. El programa lineal real utilizado para obtener el plano de separación en el espacio tridimensional es el que se describe en: [KP Bennett y OL Mangasarian: "Robust Linear

Programming Discrimination of Two Linearly Inseparable Sets", Optimization Methods and Software 1, 1992, 23-34]. Esta base de datos también está disponible a través del servidor ftp UW CS: ftp ftp.cs.wisc.edu cd math-prog/cpo-dataset/machine-learn/WDBC/

1.1.2 Referencias

WN Street, WH Wolberg y OL Mangasarian. Extracción de características nucleares para el diagnóstico de tumores de mama. Simposio Internacional sobre Imagenología Electrónica: Ciencia y Tecnología IS&T/SPIE 1993, volumen 1905, páginas 861-870, San José, CA, 1993. • OL Mangasarian, WN Street y WH Wolberg. Diagnóstico y pronóstico del cáncer de mama mediante programación lineal. Investigación Operativa, 43(4), páginas 570-577, julio-agosto de 1995. • WH Wolberg, WN Street y OL Mangasarian. Técnicas de aprendizaje automático para el diagnóstico de cáncer de mama a partir de aspirados con aguja fina. Cancer Letters 77 (1994) 163-171.

1.1.3 2.3. Lectura del conjunto de datos

```
En [1]: de sklearn.datasets importar load_breast_cancer

# Cargar el conjunto de datos de cáncer de mama
breast_cancer = load_breast_cancer ()

# Separar las variables predictoras (X) y la variable objetivo (Y)
X = breast_cancer . datos
Y = cáncer_de_mama . objetivo
```

```
En [2]: dir ( cáncer de mama )
```

```
Salida[... ['DESCRIBIR',
            'datos',
            'módulo_de_datos',
            'nombres_de_características',
            'Nombre del archivo',
            'marco',
            'objetivo',
            'nombres_objetivo']
```

1.1.4 2.2. Visualización del conjunto de datos

```
En [3]: import pandas as pd
df = pd.DataFrame(X, columns=breast_cancer.feature_names)
df
```



```
In [5]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(df, Y, stratify=Y)
print("Tamaño del conjunto de datos de entrenamiento: ", len(X_train))
print("Tamaño del conjunto de datos de pruebas: ", len(X_test))
```

Tamaño del conjunto de datos de entrenamiento: 426

Tamaño del conjunto de datos de pruebas: 143

1.1.6 2.4. Implementación de una MPNeuron más avanzada

```
In [6]: import numpy as np
from sklearn.metrics import accuracy_score

class MPNeuron:
    def __init__(self):
        self.threshold = None

    def model(self, x):
        return (sum(x) >= self.threshold)

    def predict(self, X):
        Y = []
        for x in X:
            result = self.model(x)
            Y.append(result)
        return np.array(Y)

    def fit(self, X, Y):
        accuracy = {}
        # Seleccionamos un threshold entre el número de características de entrada
        for th in range(X.shape[1] + 1):
            self.threshold = th
            Y_pred = self.predict(X)
            accuracy[th] = accuracy_score(Y, Y_pred)

        # Seleccionamos el threshold que mejores resultados proporciona
        self.threshold = max(accuracy, key=accuracy.get)
```

Seguimos teniendo un problema debido a que en nuestro conjunto de datos las características de entrada reciben valores continuos, sin embargo, nuestra MPNeuron solo procesa características de entrada con valor binario

```
In [15]: import pandas as pd
import matplotlib.pyplot as plt

## print(pd.cut([0.04, 2, 4, 5, 6, 0.02, 0.6], bins=2, labels=[0, 1]))

## plt.hist([0.04, 0.3, 4, 5, 6, 0.02, 0.6], bins=2)
## plt.show()

# Pedimos al usuario los datos separados por comas
entrada = input("Ingresa los valores separados por comas: ")

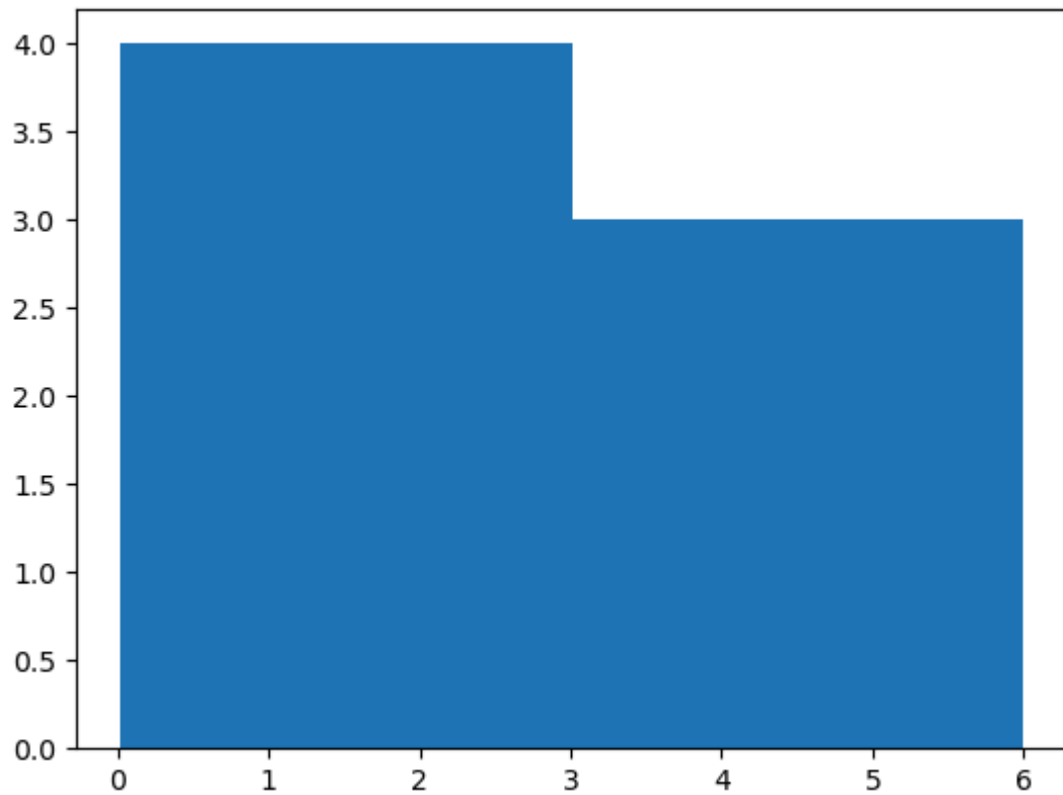
# Convertimos la entrada en una lista de números
datos = [float(x.strip()) for x in entrada.split(",")]

# Aplicamos pd.cut
```

```
print(pd.cut(datos, bins=2, labels=[0, 1]))  
  
# Graficamos el histograma  
plt.hist(datos, bins=2)  
plt.show()
```

[0, 0, 1, 1, 1, 0, 0]

Categories (2, int64): [0 < 1]



```
In [8]: # Transformamos las características de entrada a un valor binario  
X_train_bin = X_train.apply(pd.cut, bins=2, labels=[1, 0])  
X_test_bin = X_test.apply(pd.cut, bins=2, labels=[1, 0])  
  
X_train_bin
```

Out[8]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points
105	1	1	1	1	0	1	1	1
22	1	1	1	1	0	0	1	1
88	1	1	1	1	1	1	1	1
129	0	0	0	1	0	1	0	0
401	1	1	1	1	1	1	1	1
...	...	...	...	...	...	...	...	...
529	1	1	1	1	0	1	1	1
106	1	1	1	1	0	1	1	1
148	1	1	1	1	0	1	1	1
328	1	1	1	1	0	1	1	1
150	1	1	1	1	0	1	1	1

426 rows × 30 columns



```
In [9]: # Instanciamos el modelo MPNeuron
mp_neuron = MPNeuron()
# Encontramos el threshold óptimo
mp_neuron.fit(X_train_bin.to_numpy(), y_train)
```

```
In [10]: # Threshold óptimo seleccionado
mp_neuron.threshold
```

Out[10]: 26

```
In [11]: # Realizamos predicciones para ejemplos nuevos que no se encuentran en el conjunto de entrenamiento
Y_pred = mp_neuron.predict(X_test_bin.to_numpy())
```

In [12]: Y_pred

```
Out[12]: array([ True,  True,  True,  True,  True,  True, False,  True,  True,
        False,  True,  True,  True,  True,  True, False, False, False, False,
         True,  True,  True,  True, False,  True, False,  True, False,
        False, False, False, False,  True, False,  True,  True,  True,
         True, False,  True,  True,  True,  True,  True, False,  True,
        False,  True, False,  True, False,  True,  True,  True,  True,
         True,  True,  True,  True, False,  True,  True,  True,  True,
         True,  True,  True, False,  True,  True,  True,  True, False,
        False,  True,  True,  True,  True, False,  True,  True, False,
        False,  True,  True,  True,  True,  True, False, False,  True,
         True,  True,  True, False,  True,  True,  True,  True,  True,
        False, False,  True,  True, False, False,  True,  True,  True,
         True,  True,  True,  True,  True, False,  True,  True,  True,
         True,  True,  True, False,  True,  True,  True, False,  True,
        False, False, False,  True,  True,  True,  True,  True,  True,
        False,  True,  True,  True,  True,  True, False,  True])
```

```
In [13]: # Calculamos la exactitud de nuestra predicción  
precision_score ( y_test , Y_pred )
```

Salida[... 0.8041958041958042

```
En [14]: # Calculamos la matriz de confusión  
desde sklearn.metrics import confusion_matrix confusion_matrix ( y_test , Y_pred
```

Salida[... matriz([[33, 20],
[8, 82]])

[Repositorio segundo parcial](#)